Vehicle Detection - A Comprehensive Literature Review

Ethan Ernzer, Ishika Agarwal, Mete Cil, Zhenbang He*
COSC 444/544 - Computer Vision
University of British Columbia, Okanagan Campus
2025-02-24

ABSTRACT

Vehicle detection is a critical problem in computer vision with applications in autonomous driving, traffic monitoring, and surveillance. While deep learning models such as Faster R-CNN, SSD, and DETR achieve state-of-the-art performance, they require extensive labeled data and computational resources, making them impractical for some real-time applications. Traditional computer vision methods, including Scale-Invariant Feature Transform (SIFT), Histogram of Oriented Gradients (HOG), Speeded-Up Robust Features (SURF), Haar cascades, and Deformable Part Models (DPM), offer interpretable, computationally efficient solutions but they often struggle with challenges such as varying lighting conditions, and the most importantly, the diversity of vehicle types.

To address these limitations, we propose a hybrid vehicle detection pipeline that integrates HOG descriptors with a multi-class Support Vector Machine (SVM) classifier. This approach maintains the efficiency and interpretability of traditional methods while improving robustness in multi-class vehicle detection. By optimizing feature extraction and classifier design, our method aims to achieve a balance between accuracy and computational cost. This review lays the foundation for an efficient and scalable vehicle detection framework, contributing to intelligent transportation systems and automated enforcement applications.

KEYWORDS

Object Detection, Vehicle Detection, Computer Vision, Deep Learning

1 INTRODUCTION

Object recognition is a fundamental task in computer vision that involves identifying and classifying objects within images or videos. Among its various applications, vehicle detection plays a crucial role in intelligent transportation systems, traffic management, and autonomous driving. One of the key challenges in vehicle recognition lies in accurately distinguishing different types of vehicles under varying environmental conditions.

However, accurately detecting and classifying vehicles remains a challenging task due to variations in environmental conditions, occlusions, and diverse vehicle appearances. For example, poor lighting, partial vehicle occlusion, and similar visual features between different vehicle types (e.g., SUVs and trucks) can significantly reduce detection accuracy. Deep learning-based approaches, such as Faster R-CNN, SSD, and DETR, have demonstrated high accuracy in such tasks, but their reliance on large labeled datasets and high computational costs limits their feasibility for real-time applications in resource-constrained environments.

To address these challenges, this paper focuses on traditional object detection techniques, particularly feature-based and part-based approaches, such as Scale-Invariant Feature Transform (SIFT), Histogram of Oriented Gradients (HOG), Speeded-Up Robust Features (SURF), Haar wavelet-based classifiers, and Deformable Part Models (DPM). These methods have been widely used in structured environments due to their efficiency and interpretability. However, they also face challenges such as sensitivity to occlusion, high computational complexity, and reduced performance under varying lighting conditions.

A comprehensive literature review is essential to understanding the strengths and limitations of these methods, as well as identifying areas for further optimization. While deep learning-based models continue to dominate object detection research, our study aims to explore whether traditional approaches can still provide effective, lightweight, and interpretable solutions for vehicle detection.

To bridge the gap between traditional and modern methodologies, we propose a hybrid vehicle detection pipeline that integrates HOG descriptors with a Support Vector Machine (SVM) classifier. Our proposed approach builds upon the well-established Histogram of Oriented Gradients (HOG) feature descriptor, which effectively captures shape and contour information, making it suitable for vehicle recognition. Additionally, we enhance the classification capability of Support Vector Machines (SVM) by extending it to a multi-class SVM classifier, allowing the system to accurately recognize various vehicle categories, including sedans, SUVs, trucks, and buses.

The remainder of this paper is structured as follows: Section 2 reviews related work, analyzing existing traditional and deep learning methods. Section 3 outlines our proposed methodology, focusing on the HOG+SVM framework. Section 4 discusses evaluation metrics and expected outcomes, while Section 5 presents conclusions and future research directions.

2 RELATED WORK

Object detection has been a significant area of research in computer vision, supporting applications such as autonomous driving, traffic monitoring, intelligent transportation systems, and industrial automation. Over the years, detection techniques have evolved from handcrafted feature-based methods to deep learning-based models, each offering unique advantages and challenges. This section provides a comprehensive review of traditional computer vision methods, focusing on component-based representations, feature-based descriptors, and Haar wavelet techniques, and presents a brief comparison with modern deep learning approaches.

2.1 Component-Based Object Detection

Component-based methods detect and classify objects by decomposing them into individual parts. These techniques are particularly

*Names are listed in alphabetical order, with no particular ranking.

useful for handling occlusion and viewpoint variations in vehicle detection.

2.1.1 Bayesian Component-Based Detection. Zhao and Nevatia [12] proposed a component-based method for vehicle detection using aerial images. The system identifies key vehicle features, such as top boundary shape, windshield, and shadow edges, and classifies potential vehicles using a Bayesian minimum risk classifier. **Strengths:** The approach allows robustness in identifying various vehicle structures by expanding feature representations. **Weaknesses:** It is restricted to aerial imagery and requires substantial preprocessing for detection.

2.1.2 SIFT and Template Matching for Component Detection. Leung [10] introduced a two-method framework for vehicle detection at street level. The first approach uses SIFT with k-means clustering to define components, while the second employs template matching to detect parts such as tires, headlights, and side panels. **Strengths:** The SIFT-based approach performed well in handling complex street views. **Weaknesses:** The template matching approach requires handcrafted training data, making it less scalable.

2.1.3 Sparse Representation for Component-Based Detection. Agarwal et al. [1] proposed a sparse part-based representation for object detection. The method identifies interest points using the Forstner operator, clusters similar patches, and forms sparse feature vectors classified using a Sparse Network of Winnows (SNoW) learning architecture. **Strengths:** The system eliminates duplicate detections and improves accuracy. **Weaknesses:** Multiscale detection leads to higher inaccuracies, and image variations beyond the defined model reduce robustness.

2.2 Feature-Based Object Detection

Feature-based methods extract distinct visual properties from an image, such as edges, gradients, and local keypoints, to perform classification.

2.2.1 Scale-Invariant Feature Transform (SIFT). Lowe [11] introduced SIFT, a method for detecting and describing local features in an image. SIFT identifies keypoints using a Difference of Gaussian (DoG) approach and represents them as gradient histograms. **Strengths:** SIFT is robust to scale, rotation, and affine transformations. **Weaknesses:** Its computational cost is high, limiting real-time applications.

2.2.2 Histogram of Oriented Gradients (HOG). Dalal and Triggs [5] proposed HOG, which computes gradient orientations in localized regions to represent object shapes. This method is widely used for pedestrian and vehicle detection. **Strengths:** Computationally efficient and captures object contours well. **Weaknesses:** Sensitive to occlusion and viewpoint changes.

2.2.3 Speeded-Up Robust Features (SURF). Bay et al. [3] introduced SURF, an alternative to SIFT designed to reduce computation time. It uses integral images for fast keypoint detection and Haar wavelet responses for descriptor generation. **Strengths:** Faster than SIFT and suitable for real-time applications. **Weaknesses:** Less distinctive descriptors lead to higher mismatches.

2.3 Haar Wavelet and Filter-Based Detection

2.3.1 Haar-Like Features and Cascade Classifiers. Viola and Jones [16] introduced a Haar-based object detection framework that uses integral images for fast computation. Haar-like features capture intensity differences within an image and are passed to a cascade of classifiers trained using AdaBoost. **Strengths:** Highly efficient for real-time detection. **Weaknesses:** Sensitive to variations in pose, illumination, and occlusion.

2.3.2 Generalized Haar Filters with Deep Learning. Lu et al. [7] extended Haar wavelet-based detection by incorporating deep learning for lightweight vehicle detection. The method applies generalized Haar filters and sparse window generation to efficiently process large scenes. **Strengths:** Reduces the computational footprint of deep learning models. **Weaknesses:** Struggles with occlusion, unexpected orientations, and lighting variations.

2.4 Deformable Part Models (DPM)

Felzenszwalb and Huttenlocher [8] proposed Deformable Part Models (DPM), which represent objects as a set of parts arranged in a deformable structure. The model uses HOG features and a star-structured representation to detect objects under various poses and deformations. Niknejad et al.[13] use the features of vehicles are learned as a DPM through the combination of a latent support vector machine (LSVM) and HOG and tracked through a particle filter, which yields excellent accuracy. **Strengths:** Handles occlusion and variations in object structure. **Weaknesses:** Computationally expensive and complex to tune.

2.5 Comparison with Deep Learning Approaches

While this review primarily focuses on traditional methods, modern deep learning-based detectors such as Faster R-CNN, SSD, and DETR have significantly improved accuracy by leveraging convolutional neural networks (CNNs).

2.5.1 Single Shot MultiBox Detector (SSD). Liu et al. [17] introduced SSD, which replaces region proposals with a grid-based detection approach. **Strengths:** Faster than region proposal-based methods. **Weaknesses:** Less accurate for small object detection.

2.5.2 Focal Loss for One-Stage Detectors. Lin et al. [15] introduced Focal Loss, improving one-stage object detection by addressing class imbalance. **Strengths:** Focuses on hard-to-detect objects. **Weaknesses:** Increased training complexity.

2.5.3 Detection Transformer (DETR). Carion et al. [4] introduced DETR, which replaces traditional object detection pipelines with transformers. **Strengths:** Eliminates handcrafted components like anchor boxes. **Weaknesses:** High computational cost and large data requirements.

2.5.4 YOLO (You Only Look Once). YOLO[14] is a processes the entire image in a single pass through its neural network, dividing the image into a grid to simultaneously predict bounding boxes and class probabilities. **Strengths:** Significant advantages in speed, with processing rates of 45-155 frames per second. Ability to understand contextual information effectively and has strong generalization

capabilities. **Weaknesses:** Lower accuracy in object localization compared to slower two-stage detectors, especially when dealing with small objects in dense groups. Zhang et al.[18] use the Flip-Mosaic algorithm to enhance the network's perception of small targets for better vehicle detection.

2.6 Summary and Discussion

Traditional computer vision methods such as HOG, SIFT, and DPM provide interpretable, computationally efficient solutions but face challenges with occlusion, viewpoint variation, and complex backgrounds. In contrast, deep learning methods such as Faster R-CNN, SSD, DETR and YOLO outperform traditional approaches in accuracy but require extensive training data and computational power.

This review provides a comparative analysis of both paradigms, guiding the development of hybrid models that combine the efficiency of traditional methods with the accuracy of deep learning. Our proposed approach integrates HOG descriptors with an SVM classifier to enhance real-time vehicle detection while maintaining interpretability and efficiency.

3 PROPOSED METHOD

3.1 Problem Statement

Vehicle detection is a critical research area in computer vision with broad applications in transportation, safety, and security. Traditional HOG+SVM vehicle detection systems are often designed as binary classifiers, distinguishing vehicles from non-vehicles. However, real-world applications require more granular classification, differentiating between multiple vehicle types such as sedans, trucks, SUVs, and buses. This additional classification is crucial for traffic monitoring, autonomous driving, and intelligent transportation systems. Besides, many existing frameworks only support single vehicle detection which is not very useful, as multi-object detection and tracking are crucial in many practical usages like autonomous driving.

The primary challenges include:

- High intra-class variance (e.g., different sedan models may appear significantly different).
- Inter-class similarity (e.g., SUVs and trucks may share similar visual features).
- Scalability (expanding from a binary classifier to a multi-class system, and the pipeline should be easy to support new vehicle categories).
- Efficiency (Multi-class detection and multi-object detection more computation and it's critical to achieve high speed detection in some real-time application scenarios).
- Dataset (Some vehicle types are more common than others. An imbalanced dataset can cause the model to bias toward the majority class, reducing accuracy for minority vehicle types).

To address these challenges, we proposed an improved HOG+SVM framework with efficient multi-class classification capabilities as well as scalable class extension in this paper.

3.2 Core Idea

The proposed method extends the HOG+SVM pipeline by incorporating:

(1) Multi-Class SVM: Instead of a binary classifier, we use a multi-class SVM to categorize different vehicle types. We apply One-vs-All (OvA) or One-vs-One (OvO) or multilayer perceptron (MLP) classification schemes. One-vs-All scheme will be tested in priority due to its efficiency and lower dataset requirement.
(2) Optimized HOG Feature Extraction: Adjusting cell size, block size, and orientations to better capture category-specific vehicle features.
(3) Feature Fusion (Optional): Integrating additional features such as color histograms or LBP (Local Binary Patterns) to enhance discrimination.

This approach maintains the efficiency and interpretability of the original HOG+SVM model while significantly improving its classification capability.

3.3 Technical Details

In this part, we describe the input and output of our pipeline as well as the algorithm details of each components of the pipeline. The whole structure of our pipeline is illustrated in Figure 1.

Step 1: Sliding Window and Image Pyramid

To achieve multi-object and multi-scale detection. We first define a fixed-size window (e.g., 64×128 pixels) to scan the image. Move the window horizontally and vertically with a certain step size (stride). Then we will extract HOG features from each window later. Since vehicles appear in different sizes, we will also create a pyramid of scaled versions of the input image. The sliding window detection is applied to each scaled version to detect vehicles at different sizes.

Step 2: HOG Feature Extraction

HOG (Histogram of Oriented Gradients) is used to extract shape-based features from vehicle images:

Gradient Calculation: Compute gradients in the x and y directions. Cell and Block Normalization: Divide the image into small cells (e.g., 8×8 pixels) and normalize across blocks (e.g., 2×2 cells). Feature Vector Construction: Concatenate gradient histograms to form the HOG descriptor.

Step 3: Convert Binary SVM to Multi-Class classifier

Since standard SVM is a binary classifier, we extend it to multi-class classification by trying following methods:

- One-vs-All (OvA): Train separate SVM classifiers for each vehicle category (e.g., "Sedan vs. Others", "Truck vs. Others").
- One-vs-One (OvO): Train SVM classifiers for every possible pair of vehicle types (e.g., "Sedan vs. SUV", "Truck vs. Bus").
- Use an lightweight multilayer perceptron (MLP) to achieve multi-class classifier. It take preprocessed HOG features and predict multi type probabilities directly (but requires larger dataset and hyper parameters fine tuning).

Step 4: Apply Non-Maximum Suppression (NMS)

After detecting multiple bounding boxes, some may overlap significantly, representing the same vehicle multiple times. To remove redundant detections, apply Non-Maximum Suppression (NMS).

Step 5: Feature Optimization & Fine-Tuning

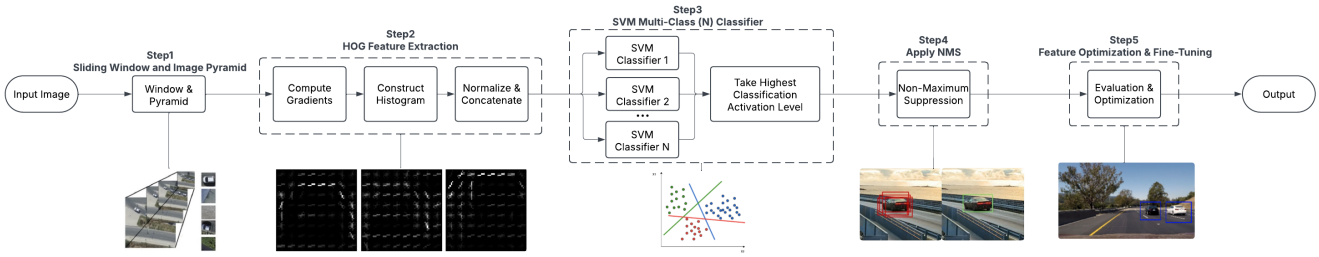

Figure 1: The pipeline of proposed vehicle detection framework.

To further improve performance, we apply:

HOG Parameter Optimization Increase orientations (from 9 to 12) for finer edge detection. Adjust cell size (e.g., 4×4 or 16×16) to better capture category-specific details. Feature Fusion (Optional Enhancement) Combine HOG with Color Histograms: Color distributions help distinguish similar vehicle shapes (e.g., yellow taxis vs. white SUVs). Integrate LBP (Local Binary Patterns): Captures fine-grained texture differences among vehicles.

Step 6: Performance Evaluation & Model Optimization

Confusion Matrix: Analyze misclassifications to identify weaknesses in feature selection. Hyperparameter Tuning: Optimize SVM or MLP's parameters to balance bias-variance tradeoff. Data Augmentation: Generate synthetic variations (e.g., rotations, brightness adjustments) to improve robustness.

3.4 Initial Experiments

We implemented the traditional HOG+SVM vehicle detection pipeline using Python and OpenCV. We trained our classifier on the Udacity car dataset[6] and tested our whole initial pipeline on the GTI vehicle image database[2], KITTI vision benchmark dataset[9]. The results of our replication are presented in Figure 2 and Figure 3.

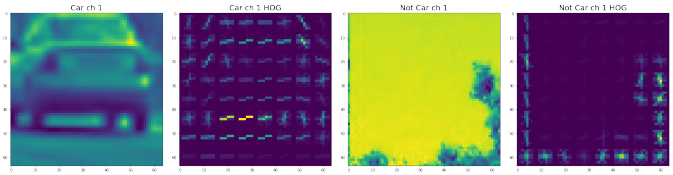

Figure 2: Extracted HOG feature of a car

We are currently working on implementing the multi-class SVM classifier and preparing multi-category vehicle dataset for training and testing.

4 PROJECTED EVALUATION METRICS

To assess the effectiveness of our proposed HOG+SVM-based vehicle detection pipeline, we define the following key evaluation metrics:

- **Accuracy (%)**: Measures the proportion of correctly classified vehicles out of the total detected vehicles.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{1}$$

Figure 3: Multi-car detection

where:
- TP = True Positives (correctly detected vehicles)
- TN = True Negatives (correctly identified non-vehicles)
- FP = False Positives (incorrectly detected vehicles)
- FN = False Negatives (missed vehicles)

- **Precision (Positive Predictive Value)**: Evaluates how many of the predicted vehicle detections are actually correct.

$$\text{Precision} = \frac{TP}{TP + FP} \tag{2}$$

- **Recall (Sensitivity)**: Assesses the proportion of actual vehicles correctly detected by the model.

$$\text{Recall} = \frac{TP}{TP + FN} \tag{3}$$

- **F1-Score**: The harmonic mean of precision and recall, providing a balanced evaluation metric.

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{4}$$

- **Mean Average Precision (mAP)**: Evaluates the model's ability to detect vehicles across different Intersection over

Union (IoU) thresholds.

$$\text{mAP} = \frac{1}{N} \sum_{i=1}^{N} AP_i \tag{5}$$

where AP_i is the Average Precision for each class, and N is the number of classes.

- **Processing Time (ms per frame)**: Measures the efficiency of the detection pipeline to ensure suitability for real-time applications.

$$\text{Processing Time} = \frac{\text{Total Time Taken (ms)}}{\text{Total Frames Processed}} \tag{6}$$

For comparison, we will evaluate these metrics against baseline deep learning models such as Faster R-CNN, SSD, and DETR to highlight computational trade-offs and detection accuracy. Our goal is to develop an interpretable and computationally efficient model while maintaining a competitive detection rate in real-world applications.

5 CONCLUSION AND FUTURE WORK

The proposed HOG + Multi-Class classifier framework significantly enhances vehicle recognition by extending from a binary classifier to a multi-category classifier. The extended system maintains the robust feature extraction capabilities of HOG while adding multi-class classification ability through modified implementation. Key improvements include:

- Ability to distinguish between multiple vehicle categories
- Capacity of detect multiple objects in a single image
- Probability scores for each vehicle class
- Maintained computational efficiency through optimized feature extraction
- Scalability to add new vehicle categories

Implementation considerations:

- Requires larger, more diverse training dataset
- May need additional computational resources for training
- Regular model updates might be necessary for new vehicle types
- Consider implementing confidence thresholds for reliable classification
- Optimized algorithm using multi-threading or parallel computing since we are performing sliding window and multi-class classification

These modifications transform the basic vehicle detector into a comprehensive vehicle classification system while building upon the proven HOG+SVM foundation.

REFERENCES

[1] Awan Agarwal and Roth. 2004. Learning to detect objects in images via a sparse, part-based representation. (2004).

[2] Jon Arróspide, Luis Salgado, and Marcos Nieto. 2012. Video analysis-based vehicle detection and tracking using an MCMC sampling framework. *EURASIP Journal on Advances in Signal Processing* 2012 (2012), 1–20.

[3] Herbert Bay, Tinne Tuytelaars, and Luc Van Gool. 2008. SURF: Speeded Up Robust Features. *Computer Vision and Image Understanding* 110, 3 (2008), 346–359.

[4] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. 2020. DETR: End-to-End Object Detection with Transformers. In *European Conference on Computer Vision (ECCV)*.

[5] Navneet Dalal and Bill Triggs. 2005. Histograms of Oriented Gradients for Human Detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 886–893.

[6] olivercameron ericrgon, macjshiggins. 2016. *Udacity Self Driving Car.* https://github.com/udacity/self-driving-car

[7] Lu et al. 2018. Generalized Haar Filter-Based Object Detection in Car Sharing Services. In *Conference on Computer Vision and Pattern Recognition (CVPR)*.

[8] Pedro Felzenszwalb and Daniel Huttenlocher. 2005. Pictorial Structures for Object Recognition. *International Journal of Computer Vision* (2005).

[9] Andreas Geiger, Philip Lenz, and Raquel Urtasun. 2012. Are we ready for Autonomous Driving? The KITTI Vision Benchmark Suite. In *Conference on Computer Vision and Pattern Recognition (CVPR)*.

[10] Leung. 2004. Component-based Car Detection in Street Scene Images. (2004).

[11] David G. Lowe. 1987. Feature-Based Object Recognition in 3D Scenes. *Artificial Intelligence* (1987).

[12] Zhao Nevatia. 2003. Car Detection in Low Resolution Aerial Images. (2003).

[13] Hossein Tehrani Niknejad, Akihiro Takeuchi, Seiichi Mita, and David McAllester. 2012. On-road multivehicle tracking using deformable object model and particle filter with improved likelihood estimation. *IEEE Transactions on Intelligent Transportation Systems* 13, 2 (2012), 748–758.

[14] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. 2016. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 779–788.

[15] Ross Girshick Kaiming He Tsung-Yi Lin, Priya Goyal and Piotr Dollár. 2018. Focal Loss for Dense Object Detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2018).

[16] Paul Viola and Michael Jones. 2004. Robust Real-Time Face Detection. *International Journal of Computer Vision* (2004).

[17] Dumitru Erhan Christian Szegedy Scott Reed Cheng-Yang Fu Wei Liu, Dragomir Anguelov and Alexander C. Berg. 2016. SSD: Single Shot MultiBox Detector. In *European Conference on Computer Vision (ECCV)*.

[18] Yu Zhang, Zhongyin Guo, Jianqing Wu, Yuan Tian, Haotian Tang, and Xinming Guo. 2022. Real-time vehicle detection based on improved yolo v5. *Sustainability* 14, 19 (2022), 12274.

Table 1: Comparison of Traditional Methods, Deep Learning, and Proposed Method

Criteria	**Traditional Methods (e.g., HOG, SIFT, DPM)**	**Deep Learning (e.g., SSD, Faster R-CNN, DETR)**	**Proposed HOG + SVM Approach**
Feature Extraction	Handcrafted (HOG, SIFT, Haar)	Automatically learned via CNNs	HOG-based handcrafted features
Computational Efficiency	High efficiency, real-time capable	Requires GPUs, slow inference	Optimized for real-time with low computational cost
Interpretability	High (features manually designed)	Low (black-box models)	High (handcrafted features and explicit decision boundary)
Accuracy on Complex Scenarios	Limited (occlusion, illumination changes)	High (large-scale datasets, complex scenes)	Moderate (improved feature selection and multi-class detection)
Training Data Requirement	Small dataset needed	Requires massive labeled datasets	Moderate dataset size needed
Robustness to Occlusion	Limited	High (deep networks can learn occlusion patterns)	Moderate (can detect occluded objects with feature engineering)
Hardware Requirements	Low (runs on CPU)	High (requires GPUs and large memory)	Low (designed for CPU efficiency)
Scalability to New Classes	Difficult (requires new feature engineering)	High (fine-tuning possible with more data)	Moderate (multi-class SVM allows expansion with fine-tuning)
Application Suitability	Real-time vehicle detection, low-power applications	High-performance object detection (self-driving cars, surveillance)	Balanced efficiency and interpretability for intelligent transport